

this reason we should try to make making the data snapshotting process as fast as possible.

Exporting a snapshot:

```
sql
BEGIN;
SELECT pgexportsnapshot(); -- '00000006-00053166-1'
-- keep the connection open
```

Do this for every connection that reads data from the table:

```
sql
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SET TRANSACTION SNAPSHOT '00000006-00053166-1'; -- use the exported snapshot

SELECT ... -- query the table
```

Data serialization

We plan to use protobuf as our serialization format because of its low space usage and fast serialization performance. While we read from PostgreSQL's default output plugin for logical decoding (pgoutput) for portability, Siphon will convert the received data into protobuf format.

The protobuf schema would express the column list as an array which would help handling database schema changes without rebuilding and redeploying the application. This means that some extra complexity of handling database column changes will need to be added to the consumer applications.

An example JSON representation of one "package" which might be pushed to the queueing system:

```
json
{
  "table": "issues",
  "schema": "public",
  "applicationidentifier": "prd.cell1.main.1",
  "events": [
    {
      "operation": "INSERT",
      "timestamp": "2018-12-10T13:45:00.000Z",
      "columns": [
        {
          "name": "id",
          "int64value": 12
        },
        {
          "name": "title",
          "stringvalue": "My issue"
        }
      ]
    }
  ]
}
```

```

    }
  ],
},
{
  "operation": "DELETE",
  "timestamp": "2018-12-10T13:45:30.000Z",
  "columns": [
    {
      "name": "id",
      "int64value": 15
    }
  ]
}
]
}

```

The package above includes two events. To prevent overloading the queueing system, the application will buffer events until a certain limit (size and time limit) and serialize the events into one package per table. For example if during a 3 second window (configurable) we receive 30 events for issues and 15 events for projects, then the application would push 2 packages to the queueing system.

Scaling

Potential bottlenecks:

- PostgreSQL logical replication slot cannot decode WAL data fast enough.
- Siphon cannot read the logical replication stream and serialize the data fast enough.

Both of these could be addressed by scaling horizontally, running multiple Siphon producer applications with different table configurations. A Siphon instance could be connected to a different PostgreSQL replica. For example:

- Application 1 replicates the main DB: issues, users, mergerequests tables.
- Application 2 replicates the main DB: events, auditevents tables.
- Application 3 replicates the CI DB: cibuilds table.

Note: One deployed Siphon application should be performant enough to process our busiest table (or table partition). In the "worst-case" scenario, a dedicated application may be required for one table (unlikely).

Note: One PostgreSQL database can support multiple Siphon applications using different logical replication slots (default is 10 slots, changing this setting requires restart).

Data volume

Some initial logical replication event volume calculations were done during the PoC phase.

Currently we see 15k-25k logical replication events (INSERT, UPDATE, DELETE) per second per

database (main, ci). Assuming that Siphon will not read logical replication events from all tables we would expect a single Siphon application to be able to handle 10k-15k logical replication events (read, parse, serialize, push to queue).

We can approximate the data volume if we pick a few tables and sample the row sizes, example:

```
sql
SELECT AVG(pgcolumnsize(t.)) AS avgrowsize from issues t where id in (select id from issues
TABLESAMPLE SYSTEM (0.001));
    avgrowsize
-----
895.0119381293470362
```

Assuming a pessimistic case where the average event size is 1 kilobyte, Siphon would handle approximately 10-15 megabytes per second of incoming network traffic from PostgreSQL. By serializing and compressing the data, the payload size could be reduced by 20% to 40%. As a result, the queuing system would process a few (packaged) events per second (assuming events are sent every 3 seconds for the configured tables) with a total payload size of a few megabytes.

Database Schema Changes

In the MVP version of the tool, introducing database schema changes may require coordination with the teams responsible for managing the ClickHouse and Snowflake databases. To mitigate the risk of accidental schema divergence between PostgreSQL and downstream database systems, we will implement tooling to ensure consistency. Additionally, we plan to add CI tests into the GitLab application repository to provide notifications when such events occur.

For the MVP, we have identified the following database tables (main database only):

- namespaces
- projects
- events
- issues
- mergerequests
- namespacedetails
- bulkimportentities
- milestones
- notes

It will eventually be possible to automatically handle database schema changes. Some ideas are described in this issue.

For now, this is how we will keep the database table schemas in sync:

- Add/Remove columns: CI test will notify us about the change.
 - ClickHouse: We add an extra migration to update the ClickHouse table schema (this can be automated).
 - Snowflake: TBD

- Alter columns (rename, precision change, etc.):
 - ClickHouse: Depending on the complexity, custom migration is needed.
 - Snowflake: TBD
- Truncate table: not supported
 - ClickHouse: Likely do a manual truncation or add an extra migration.
 - Snowflake: TBD

ClickHouse DB will be generally easier to maintain since we have similar migration tooling like we have for PostgreSQL. The migration tool is already part of the deployment process.

Consumer applications

We plan to implement a detection and retry mechanism into these applications to ensure that the database schema is not outdated.

1. When the application starts, request all database table schemas.
1. When an event package is received from the pub-sub system for a specific table, compare the received column list.
 - When the column list differs, request the table schema again (retry with backoff)
 - When the column list matches, continue.
1. Build an INSERT INTO statement with the column values.
1. Invoke the INSERT INTO statement.
1. Mark the event/item as processed for the pub-sub system.

When the column list differs and a timeout is reached, stop trying processing events data. Alerting and manual intervention will be needed.

Custom consumer application development

During the MVP phase, we don't intend to support consumer application development. We plan to have two applications developed for ClickHouse and Snowflake databases.

Authorization and data access

Access control to specific queues will be handled by the queueing system (for example NATS has advanced authorization configuration options).

Data filtering

Sensitive data such as hashes and keys must be filtered out on the producer level. Siphon's configuration should allow us to specify which columns should be skipped from the logical replication stream.

The list of database columns will come from a well-known, audited system, for example from a data catalog.

Handling red data

Handling red data and filtering out specific columns before inserting into the target database systems will be the responsibility of the consumers. The consumers might request metadata from

an external source (for example the data catalog) about the data classification of specific database columns.

Project Management

The work is tracked in the Siphon project. There is an epic for the MVP.

Language: Golang

We plan to use Golang with the following libraries:

- Reading logical replication stream: pglogrepl
- Connecting to the PostgreSQL database: pgx
- Connecting to the queueing system: nats.go (NATS is still under evaluation)
- Data serialization library: protobuf
- Compression library: TBD (we plan to use zstd)

Monitoring

The application will expose a HTTP endpoint that provides metrics data for consumption by Prometheus (setup issue, adding more metrics issue):

- Stats about the processed data (number of INSERT, DELETE, UPDATE events per table).
- Timings for each processing step (parsing, serializing).
- Buffer sizes.
- Memory usage.
- Info about the replication lag (PostgreSQL side). Keep track of the not consumed data volume from the replication slot.

Some of this data should also be logged to integrate the application with our logging system (Kibana).

Encryption

Connections to the database and the pub-sub (queueing) system will utilize secure TLS connections to ensure data security during transmission.

Encrypting and decrypting messages within the Siphon application will not be necessary if the compute nodes running the pub-sub system have full-disk encryption enabled.

Requirements:

- TLS/SSL connectivity to:
 - PostgreSQL
 - ClickHouse
 - Snowflake
 - The pub-sub system
- Full-disk encryption for the pub-sub system's storage.

NOTE:

For more context, see the discussion issue.

Failure scenarios

PostgreSQL connection is closed

The application will retry a few times and then exits.

In case the initial data snapshot connection is closed, the whole snapshot process needs to be cleaned up. This error would not be recoverable and the snapshot process would need to be started again from the beginning (drop all exported data).

Queueing system is not available

The application will retry a few times and then exits.

Payload rejected by the queueing system because the topic/queue is not available

Use a retry mechanism.

The application crashes

The environment should ensure that the application is restarted.

The Siphon pod is terminated

The environment should detect this and start a new instance. The producer application does not persist data on the local disk.

Runtime risks

Logical replication on the PostgreSQL database server can consume a significant amount of resources. To minimize the impact on the primary database, we plan to use replicas for reading the logical replication stream and taking the initial data snapshot.

Issues and mitigations

Siphon is down or not consuming WAL data

- Problem: Accumulation of WAL files on the replica, leading to high disk usage.
- Mitigation:
 - Monitor replication lag and the number of WAL files.
 - Define a threshold ("point of no return") to drop the replication slot if necessary.

Database replica is gone or replication slot is dropped

- Problem: Replication cannot be resumed in a consistent manner.
- Mitigation:
 - Perform a full re-sync (initial data snapshot) of the data.
 - Ensure consumers and downstream systems can rebuild/restart the sync process from scratch,

including necessary cleanup steps.

Performance issues on PostgreSQL

- Problem: Logical decoding lag growing · unable to keep up with rates of writes.
- Mitigation:
 - Scale horizontally, use multiple Siphon applications with different replica servers.

Consistency

- Problem: Missing WAL record(s) due to a bug or switchover/failover.
- Mitigation:
 - Assess the impact on the target database system and decide whether we need to resync the databases.

Future considerations

PostgreSQL 17 is expected to introduce a feature to synchronize logical replication slots. This feature could prevent failures that require a full data resync, making logical replication more robust.

Deployment

External dependencies

Producer application:

- Connection to a PostgreSQL server configured for logical replication (PostgreSQL v14+).
- Connection to a queueing system.

Consumer application:

- Connection to a queueing system (e.g., NATS).
- Connection to the target system (e.g., ClickHouse, Snowflake).

PostgreSQL Server(s)

With PostgreSQL 16, it's possible to set up logical replication on database replicas. This allows Siphon to extract data with low latency from PostgreSQL without affecting the primary database or other read-only replicas used by the main GitLab application.

Optionally, the snapshotting process may use different PostgreSQL server. This is especially important for larger GitLab instances (GitLab.com). For GitLab.com we plan to use a physical standby working on streaming replication where we disable the hotstandbyfeedback setting.

This allows us to take initial data snapshot without affecting the cluster (long transaction, blocking vacuum).

Infrastructure needs:

- One node for running the producer application (after the MVP we will run multiple applications).
- One or more nodes for running the consumer applications (1 application per target database).
- A cluster for running the queueing system (TBD).

Rollout

The tool will be built to allow gradual addition of tables via a configuration file. When a new table is added, the system will take an initial data snapshot as the first step and then it continues with the changes captured via the logical replication stream.

After MVP Steps

- Automated schema detection from the GitLab application.
- Multiple instances of the application running (CI, Main, Sec databases).
- A coordinator application to dynamically alter producer configurations.
- Dedicated, self-managed support.

SECTION: engineering/architecture/design-documents/slsa_level_3/_index.md

{{< engineering/design-document-header >}}

Summary

This document outlines the technical vision, principles, and key architectural decisions for implementing SLSA Level 3 compliance within GitLab CI/CD pipelines. The solution will provide a reusable, modular, and secure way to generate, sign, and verify provenance for artifacts while hardening pipeline identity and build infrastructure.

Proposal

We propose a phased implementation of SLSA Level 3 compliance across GitLab CI/CD pipelines using modular and reusable components. Each phase addresses a critical step:

1. In-Pipeline Provenance Generation and Verification using Sigstore: Generate and verify provenance attestation within the pipeline.
1. Generate Provenance Statement in Control Plane: Shift provenance generation from Runner to GitLab Rails backend to enhance trust.
1. Sign Provenance Statement in Control Plane: Move signing operations to dedicated service for better security.
1. KMS Integration for out-of-Pipeline Signing: Enable external, KMS-based artifact signing for enhanced security and compliance.
1. Hardening Pipeline Identity: Strengthen runner identity and build trust into the infrastructure.

This phased approach ensures an MVP can be delivered early, with incremental security and compliance enhancements added over time.

Goals

- Embed GitLab-specific platform data (pipeline variables, commit IDs) into provenance for traceability.
- Support out-of-pipeline signing via secure KMS or HSM, isolating signing keys from build environments.
- Strengthen runner identity to provide trustworthy attestation of build provenance.
- Align with SLSA Level 3 compliance requirements while minimizing disruption to existing workflows.
- Ensure security, scalability, and ease of adoption across GitLab environments.

Non-Goals

- Achieving SLSA Level 4 compliance, which requires isolated and verifiable builds (future consideration).
- Collecting dependencies of all possible programming ecosystems (focus on key ecosystems first).
- Replacing GitLab's existing artifact storage and distribution mechanisms.

Terminology/Glossary

- SLSA: Supply-chain Levels for Software Artifacts, a framework for improving supply chain security.
- Provenance predicate: Metadata that describes how an artifact was built, including the source code, dependencies, and environment.
- Provenance statement: Document that binds a provenance predicate to a software artifact.
- Provenance attestation: Envelope that combines a provenance statement with a signature.
- Sigstore: An open-source tool for signing, verifying, and storing software artifacts securely (cosign and gitsign).
- OIDC Token: Short-lived, identity-based tokens issued by GitLab CI for secure signing.
- Runner: A build agent that executes GitLab CI/CD pipeline jobs.
- KMS: Key Management Service, an external system to securely manage cryptographic keys.
- HSM: Hardware Security Module, hardware-based systems for secure key storage and signing.
- VSA: Verification Summary Attestation, an attestation that an artifact has been verified to meet certain requirements.
- GitLab Rails backend: The GitLab Rails backend that provides a trusted environment for security-critical operations, separate from the build environment.
- Signing Service: A dedicated service within the GitLab infrastructure responsible for SLSA statement signing operations, isolated from the CI/CD execution environment.

Assumptions

- Provenance Generation: Use Sigstore Cosign (CLI, client, or port) to generate provenance attestations.
- OIDC-based keyless signing: Use Signstore Fulcio for OIDC-base keyless signing.
- Transparency log: Use Sigstore Rekor transparency log.
- Signing Methods:
 - In-pipeline signing via OIDC-based short-lived credentials for fast MVP.
 - Out-of-pipeline signing via KMS for long-term secure artifact signing.
- Runner Hardening: Explore options for strong runner identity using hardware-based solutions

(TPM, secure enclaves).

Design Details

High Level Architecture

mermaid

flowchart TD

%% Define styles for improved visual appearance

classDef phaseStyle fill:#f9f9f9,stroke:#333,stroke-width:2px,rx:10px,ry:10px

classDef componentStyle fill:#e1ebff,stroke:#4b6bdc,stroke-width:1px,rx:5px,ry:5px

classDef storageStyle fill:#ffe6cc,stroke:#d79b00,stroke-width:1px,rx:5px,ry:5px

classDef serviceStyle fill:#d5e8d4,stroke:#82b366,stroke-width:1px,rx:5px,ry:5px

classDef signatureStyle fill:#fff2cc,stroke:#d6b656,stroke-width:1px,rx:5px,ry:5px

classDef securityStyle fill:#f8cecc,stroke:#b85450,stroke-width:1px,rx:5px,ry:5px

classDef controlPlaneStyle fill:#e1d5e7,stroke:#9673a6,stroke-width:1px,rx:5px,ry:5px

subgraph BuildEnvironment["Build Environment"]

Runner["Runner"]

BuildJob["CI/CD Build Job"]

Artifacts["Job Artifacts"]

end

subgraph FutureWork["Dependency tracking"]

VirtualRegistry["Virtual Registry
(Dependency Proxy)"]

Dependencies[(Package & Container
Dependencies)]

end

subgraph ControlPlane["Control Plane"]

subgraph GenerateProvenanceInControlPlane["Phase 2: Generate Provenance in Control Plane"]

RailsBackend["GitLab Rails Backend"]

DB[(GitLab Database)]

end

subgraph SignProvenanceInControlPlane["Phase 3: Sign Provenance in Control Plane"]

GlgoService["glgo Service
(Signing Service)"]

end

Rekor["Transparency Log
(Rekor)"]

PermanentAttestation["Permanent Signed
Attestation"]

end

subgraph Phase4["Phase 4: Get private key from KMS"]

ExternalKMS["External KMS"]

end

%% Relationships between components with labeled edges

Runner -->|"Request job payload
with proof of identity"| RailsBackend

RailsBackend -->|"Return job payload"| Runner

Runner -->|"Executes"| BuildJob

BuildJob -->|"Upload"| Artifacts

BuildJob -->|"Request Dependencies"| VirtualRegistry

VirtualRegistry <-->|"Fetch/Track"| Dependencies

%% Phase 1 flow for early implementation

VirtualRegistry -->|"Provide Dependency Data"| RailsBackend
RailsBackend <-->|"Query job parameters"| DB

RailsBackend -->|"Send Provenance
Statement"| GlgoService
GlgoService -.->|"Future Integration"| ExternalKMS
GlgoService -->|"Return Signed
Attestation"| RailsBackend
GlgoService -->|"Publish Attestation
Digest"| Rekor
RailsBackend -->|"Upload"| PermanentAttestation

%% Apply styles
class FutureWork phaseStyle
class Phase4 phaseStyle
class Artifacts,DB,Dependencies storageStyle
class VirtualRegistry,GlgoService serviceStyle
class ProvenanceSigner,TempSignedAttestation,PermanentAttestation,Rekor,ExternalKMS
signatureStyle
class RailsBackend controlPlaneStyle

Phase 1: In-Pipeline Provenance Generation and Verification using Sigstore

- Generate provenance attestations using Sigstore tools (cosign).
- Leverage GitLab CI's OIDC tokens for secure and short-lived credentials.
- Verify provenance attestations and generate Verification Summary Attestations (VSA).
- Build reusable GitLab CI components that can be easily included in pipelines.

Phase 2: Generate Provenance Statement in Control Plane

- Move provenance statement generation to GitLab Rails backend.
- Every field of the provenance is generated or verified in the trusted control plane.
- Generate provenance statements that are unforgeable.

Phase 3: Out-of-Pipeline Signing

- Move signing operations from CI/CD pipelines to a signing service running on the backend.
- Enhance security by isolating signing operations from build environment.
- Provide centralized management of signing processes.
- Ensure clean separation between build and signing trust boundaries.

Phase 4: KMS Integration for out-of-pipeline Signing

- Enable integration with external KMS (AWS KMS, Google KMS) or HSM solutions.
- Use long-term signing keys stored securely outside the pipeline.
- Support multiple key management solutions to accommodate various enterprise environments.

Phase 5: Hardening Pipeline Identity

- Introduce strong runner identity using secure hardware (TPM, HSM, secure boot).
- Embed runner identity into the provenance metadata.
- Ensure that the runner environment is verified and trusted.

Follow-up Work: Enhanced Data Collection

- Integrate tools to collect granular build metadata (go mod graph for Go, Maven dependency trees for Java).
- Use GitLab's Virtual Registry (formerly Dependency Proxy) to track resolved dependencies requested by CI/CD build jobs.
- Update provenance structure to include enriched metadata.

Implementation Details

Sigstore and GitLab OIDC Integration

- How Sigstore Will Be Used:
 - Sigstore tools, specifically cosign, will be leveraged to sign and verify provenance.
 - cosign can utilize GitLab CI's OIDC integration to securely authenticate the job and issue short-lived credentials. These credentials will sign the provenance file.
 - The OIDC token provided by GitLab is scoped to the running pipeline job, making it ephemeral and secure.
- GitLab OIDC Integration Workflow:
 - GitLab generates an OIDC token in the CI component and exposes it as an environment variable.
 - Sigstore's cosign uses the OIDC token to authenticate the GitLab CI job with Sigstore's transparency log (Rekor).
 - Sigstore validates the identity and grants signing capability for the duration of the job.
 - cosign generates a signed provenance file (JSON format) and uploads it to GitLab's artifacts store.
- OIDC Configuration in GitLab:
 - Enable GitLab OIDC support by using the existing ID Token feature.
 - Use GitLab CI/CD's environment variables to expose tokens and necessary metadata.

GitLab CI Components

<details>

<summary>Provenance Signer Component</summary>

The provenance signer component will abstract away the complexity of provenance generation and signing. It will be implemented as a GitLab CI Component using a template YAML file.

Component Overview

- Input Variables:
 - TARGETARTIFACT: Path to the artifact or build output.
 - BUNDLEFILE: Path to generate the bundle file. This contains everything needed to verify the artifact.
 - RUNNERMETADATFILE: This is the default filename when artifacts aren't explicitly named.
- Output:
 - Signed provenance file uploaded as a pipeline artifact.

Example reusable Component YAML

yaml

.gitlab/components/provenance-signer.yml

component:

inputs:

variables:

TARGETARTIFACT: "" Path to the artifact

BUNDLEFILE: "provenance.json" Output bundle file

RUNNERMETADATAFILE: "artifacts-metadata.json" This is the default filename when artifacts aren't explicitly named

idtokens:

GITLABOIDCTOKEN:

aud: sigstore

variables:

REKORSERVER: "https://rekor.sigstore.dev"

FULCIOSERVER: "https://fulcio.sigstore.dev"

image: alpine:latest

beforescript:

- apk add --update cosign jq

script:

- echo "Fetching GitLab Runner metadata..."

- export RUNNERMETADATA=\$(jq -c . \${RUNNERMETADATAFILE})

- echo "Generating predicate for \${TARGETARTIFACT}..."

- echo "\${RUNNERMETADATA}" | jq -c .predicate > predicate.json

- echo "Attesting provenance for \${TARGETARTIFACT}..."

- cosign attest-blob --predicate predicate.json \
 --type slsaprovenance1 \
 --oidc-issuer "\${CISERVERHOST}" \
 --fulcio-url "\${FULCIOSERVER}" \
 --rekor-url "\${REKORSERVER}" \
 --identity-token "\${GITLABOIDCTOKEN}" \
 --bundle "\${BUNDLEFILE}" \
 --new-bundle-format \
 "\${TARGETARTIFACT}"

- echo "Performing self-verification to ensure provenance is valid..."

- cosign verify-blob-attestation --type slsaprovenance1 \
 --bundle "\${BUNDLEFILE}" \
 --certificate-identity-regexp "." \
 --certificate-oidc-issuer "\${CISERVERURL}" \
 "\${TARGETARTIFACT}"

- echo "Self-verification successful! Provenance is valid."

artifacts:

```
paths:
  - ${BUNDLEFILE}
expirein: 7d
```

</details>

<details>
<summary>Provenance Verifier Component</summary>

The provenance verifier component verifies attestations and generates VSAs. It will be implemented as a GitLab CI Component using a template YAML file.

Component Overview

- Input Variables:
 - BUNDLEFILE: Path to the bundle file that contains the provenance.
 - VERIFICATIONSUMMARYFILE: Path to generate the verification summary attestation.
 - RESOURCEURL: Full URL to the published artifact.
 - POLICYURL: URL to the policy used for verification.
- Output:
 - Verification summary attestation uploaded as a pipeline artifact.

Example reusable Component YAML

```
yaml
.gitlab/components/provenance-verifier.yml
component:
  inputs:
    variables:
      BUNDLEFILE: "cosign-bundle.json" Path to the bundle file
      VERIFICATIONSUMMARYFILE: "verificationsummary.json" Output verification summary file
      RESOURCEURL: "" Full URL to the published artifact
      POLICYURL: "https://gitlab.com/slsa-vsa-policy/v1" Default policy URL

  idtokens:
    GITLABOIDCTOKEN:
      aud: sigstore

  variables:
    REKORSERVER: "https://rekor.sigstore.dev"
    FULCIOSERVER: "https://fulcio.sigstore.dev"
    VERIFIERID: "https://gitlab.com/verifier"
    VERIFIERNAME: "GitLab Verification Pipeline"
    DOWNLOADEDARTIFACT: ".tmp/downloadedartifact"

  image: alpine:latest

  beforescript:
    - apk add --update cosign jq curl
```

- mkdir -p .tmp

script:

```
- echo "Downloading artifact from ${RESOURCEURL}..."
- mkdir -p $(dirname ${DOWNLOADEDARTIFACT})
- curl -L -o ${DOWNLOADEDARTIFACT} ${RESOURCEURL}

- echo "Calculating artifact digest..."
- ARTIFACTDIGEST=$(sha256sum ${DOWNLOADEDARTIFACT} | cut -d ' ' -f 1)

- echo "Downloading policy from ${POLICYURL}..."
- POLICYFILE=".tmp/policy.json"
- |
  if ! curl -L -f -o ${POLICYFILE} ${POLICYURL}; then
    echo "ERROR: Failed to download policy file from ${POLICYURL}"
    exit 1
  fi

- echo "Calculating policy digest..."
- POLICYDIGEST=$(sha256sum ${POLICYFILE} | cut -d ' ' -f 1)
- echo "Policy digest: ${POLICYDIGEST}"

- echo "Verifying signed provenance against downloaded artifact..."
- cosign verify-blob-attestation --type slsaprovenance1 \
  --bundle ${BUNDLEFILE} \
  --certificate-identity-regexp "." \
  --certificate-oidc-issuer ${CISERVERURL} \
  ${DOWNLOADEDARTIFACT}
- RESULT="PASSED" TODO: verify the provenance against the policies

- echo "Generating verification summary for artifact..."
- mkdir -p $(dirname ${VERIFICATIONSUMMARYFILE})
- jq -n --arg policyUrl "${POLICYURL}" --arg result "${RESULT}" \
  --arg verifierId "${VERIFIERID}" \
  --arg timeVerified "$(date -u +%Y-%m-%dT%H:%M:%SZ)" --arg resourceUri
"${RESOURCEURL}" \
  --argjson verifiedLevels '["SLSAL3"]' --arg sha256 "${ARTIFACTDIGEST}" \
  --arg bundleFilePath "${BUNDLEFILE}" --arg bundleFileHash "$(sha256sum ${BUNDLEFILE}
| cut -d ' ' -f 1)" \
  --arg policyDigest "${POLICYDIGEST}" \
  --arg slsaVersion "1.0" '{
"type": "https://in-toto.io/Statement/v1",
"subject": [{
"name": $resourceUri,
"digest": { "sha256": $sha256 }
}],
"predicateType": "https://slsa.dev/verificationsummary/v1",
"predicate": {
"verifier": {
"id": $verifierId
```

```

    },
    "timeVerified": $timeVerified,
    "resourceUri": $resourceUri,
    "policy": {
      "uri": $policyUrl,
      "digest": {
        "sha256": $policyDigest
      }
    },
    "inputAttestations": [
      {
        "uri": $bundleFilePath,
        "digest": {
          "sha256": $bundleFileHash
        }
      }
    ],
    "verificationResult": $result,
    "verifiedLevels": $verifiedLevels,
    "dependencyLevels": {
      "SLSAL3": 3
    },
    "slsaVersion": "1.0"
  }
}' > "${VERIFICATIONSUMMARYFILE}"

```

```

- echo "Verification summary generated at ${VERIFICATIONSUMMARYFILE}"
- jq . ${VERIFICATIONSUMMARYFILE}

```

```

- echo "Signing the verification summary attestation..."
- cosign attest-blob --predicate "${VERIFICATIONSUMMARYFILE}" \
  --type slsaverificationsummary \
  --oidc-issuer "${CISERVERHOST}" \
  --fulcio-url "${FULCIOSERVER}" \
  --rekor-url "${REKORSERVER}" \
  --identity-token "${GITLABOIDCTOKEN}" \
  --bundle "${VERIFICATIONSUMMARYFILE}.bundle" \
  "${DOWNLOADEDARTIFACT}"

```

```

- echo "VSA signed and stored at ${VERIFICATIONSUMMARYFILE}.bundle"

```

```

- |
  if [ "$RESULT" == "FAILED" ]; then
    echo "Policy verification FAILED. Exiting with error."
    exit 1
  fi

```

artifacts:
 when: always
 paths:

```
- ${VERIFICATIONSUMMARYFILE}
- ${VERIFICATIONSUMMARYFILE}.bundle
expirein: 7d
```

```
allowfailure: true
```

```
</details>
```

```
<details>
```

```
<summary>Example: Adding the Components to a Pipeline</summary>
```

Here's how a project would integrate the reusable component into their .gitlab-ci.yml pipeline.

Pipeline YAML Example

```
yaml
```

```
stages:
```

- build
- provenance
- publish
- verify

```
variables:
```

```
  RUNNERGENERATEARTIFACTSMETADATA: "true"
```

```
  RUNNERMETADATAFILE: "artifacts-metadata.json" This is the default filename when artifacts
aren't explicitly named
```

```
buildartifact:
```

```
  stage: build
```

```
  script:
```

- echo "Building artifact..."
- mkdir -p dist
- echo "Example artifact content" > dist/example-artifact.txt

```
  artifacts:
```

```
    paths:
```

- dist/

```
    expirein: 7d
```

```
generateprovenance:
```

```
  stage: provenance
```

```
  needs: ["buildartifact"]
```

```
  component: .gitlab/components/provenance-signer.yml
```

```
  variables:
```

```
    TARGETARTIFACT: "dist/example-artifact.txt"
```

```
    BUNDLEFILE: "dist/provenance.json"
```

```
    RUNNERMETADATAFILE: "${RUNNERMETADATAFILE}"
```

```
publishartifact:
```

```
  stage: publish
```

```
needs: ["generateprovenance"]
script:
  - echo "Publishing artifact to package registry..."
  - |
    ARTIFACTURL=$(curl --header "JOB-TOKEN: ${CIJOBTOKEN}" \
      --upload-file dist/example-artifact.txt \
      "${CI_APIV4_URL}/projects/${CI_PROJECT_ID}/packages/generic/artifacts/1.0.0/example-
artifact.txt" \
      | jq -r '.location')
  - echo "ARTIFACTURL=${ARTIFACTURL}" >> publish.env
artifacts:
  reports:
    dotenv: publish.env
```

```
verifyprovenance:
  stage: verify
  needs: ["publishartifact"]
  component: .gitlab/components/provenance-verifier.yml
  variables:
    BUNDLEFILE: "dist/provenance.json"
    VERIFICATIONSUMMARYFILE: "dist/verificationsummary.json"
    RESOURCEURL: "${ARTIFACTURL}"
    POLICYURL: "https://gitlab.com/my-policy"
```

</details>

Pipeline Workflow Explanation

- Build Artifact Stage (buildartifact):
 - Builds the artifact (binary, container image, etc.).
 - Saves the artifact as a pipeline artifact.
- Provenance Generation Stage (generateprovenance):
 - Uses the provenance-signer component to:
 - Generate a detailed provenance predicate with build metadata.
 - Calculate the artifact's digest for inclusion in the provenance statement.
 - Sign the provenance using Sigstore's cosign with GitLab's OIDC token.
 - Perform self-verification to ensure the provenance is valid.
 - Uploads the signed provenance as a job artifact.
- Publish Artifact Stage (publishartifact):
 - Publishes the artifact to a registry or repository.
 - Captures the published artifact's URL for use in verification.
 - This stage separates build/sign from verification, ensuring a true separation of concerns.
- Provenance Verification Stage (verifyprovenance):
 - Uses the provenance-verifier component to:
 - Download the published artifact from its URL.
 - Download the required verification policy.
 - Verify the signed provenance against the downloaded artifact.
 - Check SLSA L3 requirements in the attestation.
 - Generate a Verification Summary Attestation (VSA).

- Sign the VSA using Sigstore's cosign with GitLab's OIDC token.
- Upload the VSA as a job artifact.
- Fails the job if verification fails, but allows the pipeline to continue.

Security Considerations

- Ephemeral OIDC Tokens:
 - The ID token is short-lived and scoped to the current job.
 - This ensures it cannot be reused outside the pipeline execution context.
- Artifact and Provenance Storage:
 - Use GitLab's artifact storage to securely store both the build artifact and the signed provenance file.
 - Artifacts are automatically managed and can be expired after a specified time.
- Isolation:
 - If using shared runners, ensure sandboxed environments (ephemeral containers).
 - Self-hosted runners should follow security best practices to prevent token leakage.
- Dependency Management:
 - Pin specific versions of Sigstore tools (cosign) to prevent supply chain attacks.
- CI Variables:
 - CI Variables will be included in signed provenance file. But will follow the Visibility setting where Masked or Masked and hidden variables will not store the value, only the key.
- Separation of Concerns:
 - The separation of provenance generation and verification ensures that verification is truly independent.
 - The VSA is generated by the verifier component, providing a clear chain of trust.

Component Maintenance and Scalability

- Publish the GitLab CI component in a versioned Git repository to ensure teams can pull stable versions.
- Provide clear documentation and examples for adoption.
- Extend the component in later phases to include additional metadata collection and signing enhancements.

Decisions

- 001: Verification Component - Verify SLSA provenance attestations in a dedicated CI/CD component.
- 002: Provenance Generation Location - Generate SLSA provenance statements in the GitLab Rails backend.

SECTION: engineering/architecture/design-
documents/slsa_level_3/decisions/001_verification_component.md

Context

GitLab is implementing SLSA Level 3 compliance features for CI/CD pipelines. A key aspect of this implementation is the generation and verification of attestations that provide evidence of

build provenance. The initial design documented the creation of a reusable GitLab CI component for generating and signing SLSA-compliant provenance, but lacked a clear separation between the attestation generation and verification processes.

SLSA (Supply-chain Levels for Software Artifacts) attestations provide cryptographic proof about how artifacts were created. Verification Summary Attestations (VSAs) are separate documents that record the results of verifying these attestations against a policy.

Problem Statement

The current design has the following issues:

- Lack of separation of concerns: The same component that generates and signs the provenance attestation also creates the Verification Summary Attestation (VSA).
- Verification is not truly independent: When the same component generates both the attestation and VSA, there's no true validation that the attestation meets SLSA L3 requirements.
- Pipeline structure doesn't reflect real-world usage: In production environments, artifact publishing and verification often occur in separate stages or even separate pipelines.

Options Considered

Option 1: Single Unified Component

Description: Keep a single component that handles both attestation generation and verification.

Pros:

- Simpler implementation
- Fewer components to maintain
- Less pipeline overhead

Cons:

- No true separation of concerns
- The VSA doesn't represent an independent verification.
- It doesn't align with enterprise CI/CD practices where build, release, and verification are often separate processes.

Option 2: Split Functionality into Separate Components

Description: Create two distinct components:

- A provenance-signer component that generates a provenance attestation.
- A provenance-verifier component that verifies the provenance attestation and generates a VSA.

Pros:

- Clear separation of concerns
- The VSA truly represents an independent verification
- Aligns with enterprise CI/CD practices

- Better security through independent verification
- Enables using different environments or permissions for signing vs verification
- Enables verification out of the build pipeline and after publishing the artifact

Cons:

- More components to maintain
- Slightly more complex pipeline configuration
- Additional pipeline stage required

Decision

We have decided to implement Option 2: Split Functionality into Separate Components. This decision provides several advantages:

1. Security Improvement: By separating the generation and verification of attestations, we enable a more robust security model where the verification process is truly independent.
1. Real-world Alignment: This structure better aligns with enterprise CI/CD practices where artifacts might be built, signed, published, and later verified in separate processes.
1. Future Flexibility: The separated components provide flexibility for future enhancements:
 1. Different identities could be used for signing vs. verification
 1. Verification could happen in different environments (e.g., during deployment)
 1. Different policies could be applied during verification
1. Proper VSA Semantics: The Verification Summary Attestation becomes what it claims to be - a summary of an independent verification process, not just an extension of the attestation generation.
1. Clear Responsibility Boundaries: Each component has a clear, singular responsibility which aligns with good software design principles.

Open Questions

- Should the verification component be configurable to verify against different SLSA levels?
- What specific SLSA L3 requirements should be checked during verification?
- How will we handle verification failures in downstream processes?
- What are the default policies? What do they assert of the provenance predicate?
- Are policies defined in a language such as Rego?
- Should we support user-defined policies?

SECTION: engineering/architecture/design-
documents/slsa_level_3/decisions/002_provenance_generation_location.md

Context

SLSA (Supply chain Levels for Software Artifacts) is a security framework that helps ensure the integrity of software artifacts. As part of implementing SLSA Level 3 compliance, GitLab needs to generate provenance statements for artifacts produced by CI/CD pipelines. These statements contain metadata about how an artifact was built, including repository URL, commit SHA, build ID, runner ID, CI/CD variables, and artifact digests.

Currently, SLSA 1.0 provenance statements are generated by Runner helpers. However, this approach doesn't meet SLSA Level 3 requirements, which specify that provenance must be generated by the control plane (within the trust boundary) and not by a tenant of the build platform.

This ADR documents the decision on where to generate SLSA provenance statements within GitLab's architecture.

Options Considered

1. Runner Helper (Current Implementation)

Pros:

- Already implemented.
- Can track files read and written by the build job (not implemented).
- Easy setup, enabled by a single CI/CD variable.

Blockers:

- Not in control plane.

2. Runner

Pros:

- Can track files read and written by the build job.

Cons:

- Can't trust the runner identity. Self-reported runner ID could be falsified.

Blockers:

- Not in control plane.
- Has no access to the artifacts, and can't get their digests. Artifacts are uploaded by Runner helpers.

3. GitLab Rails Backend

Pros:

- In control plane.
- Easy deployment, maintenance, and monitoring.
- Easy setup: provenance generation could be enabled in GitLab UI
- Compliance: provenance can be enforced regardless of what's defined in CI/CD configuration.
- Security: Rails backend can check the runner's identity.
- Already has access to necessary metadata (job payload and artifact digests).

Cons:

- Cannot track files read and written by the build job.

4. GitLab Workhorse

Pros:

- Already generates artifacts metadata, which are similar to provenance statement.
- Has access to the artifacts. Workhorse uploads them to object storage, generates their digests, and posts artifact metadata to the Rails backend.

Blockers:

- Cannot provide all necessary metadata.
- Cannot verify runner identity in a trusted way.

5. CI/CD Component

Pros:

- Visibility: advertised in the CI/CD catalog
- Maintenance: new releases can be published at any time

Cons:

- Air gap: CI/CD components don't ship with GitLab, requiring extra setup.
- Complex setup for users. Users must configure their CI/CD pipelines in order to pass build metadata over to the provenance generation component.

Blockers:

- Not in control plane.

6. New Service

Pros:

- Clear trust boundaries.
- Independent scaling.

Cons:

- New service to maintain.
- Additional infrastructure required.
- Significant integration effort.

Blockers:

- No direct access to provenance metadata.

Decision

We will generate SLSA provenance statements in the GitLab Rails backend.

This is the only option that satisfies the critical SLSA Level 3 requirement that provenance must be generated within the control plane, and that provides all necessary build metadata.

The Rails backend has access to all necessary metadata to generate complete provenance statements, including:

1. build ID (from job record)
2. runner ID (verified by the backend)
3. build ID (from job record)
4. runner ID (verified by the backend)
5. repository URL and commit SHA (from job payload)
6. CI/CD variables (from job payload)
7. artifact digests (stored in ci database accessible from backend)

The performance impact of generating provenance statements is expected to be minimal compared to other operations performed by the Rails backend when jobs complete.

If needed, provenance generation can be handled by background jobs to further minimize performance impact.

Consequences

Positive

- SLSA Level 3 compliance is achievable.
- Provenance generation is integrated with existing GitLab infrastructure.
- User experience is simplified through GitLab UI configuration.
- No additional services or infrastructure required.

Negative

- Cannot track files read by the build job.
Tracking these files would make it possible to generate the resolvedDependencies field.
However, SLSA L3 requirements state that completeness of that field is best effort.
- Cannot track files written by the build job.
Tracking these fields would make it possible to generate the byProducts field.
However, this field isn't required.

Future Considerations

- The signing of provenance statements is being addressed separately in issue 537060
- Future enhancements may explore ways to capture resolvedDependencies and byProducts, potentially through integration with the runner while still generating the provenance in the control plane.

Related Links

- SLSA 1.0 Requirements
- SLSA 1.0 Provenance

- Issue 537049: Architectural Decision: Where do we generate SLSA provenance?

SECTION: engineering/architecture/design-documents/ssh_certificates/_index.md

{{< engineering/design-document-header >}}

Summary

On GitLab.com customers obtain their own top-level group (later organization). In comparison to self-managed, they have to manage organization-wide settings at this level.

Currently, the provided Git access control options on SaaS (SSH, HTTPS) rely on credentials (access tokens, SSH keys) setup in the user profile. As the user profile is out of control of the organization, there is no way for a customer to assess whether the key is kept confidential or whether the expiry date is meeting policies. Also, there's very little that can be done for damage control in case the keys are leaked as well as a customer cannot enforce MFA on Git access flows.

Customers may have processes in place, where developers on a daily basis can, via MFA, request a temporary SSH certificate which gives them access to internal systems. To enable the same way of working on SaaS, we would need a way to share public Certificate Authority (CA) files with GitLab.com SaaS for the purpose of Git access control.

Motivation

- Enable users to share public Certificate Authority (CA) files with GitLab.com SaaS for the purpose of Git access control.
- Fill the product gap between GitLab and competitive products that already support authentication via SSH certificates.

Goals

This document proposes an architectural design to implement functionality to satisfy the following requirements:

- The public key of the CA file (CA.pub) that is used to issue certificates can be added to a group.
- A certificate issued by the CA can be used to get Git access to projects of the group and its ancestors.
- The certificate cannot be used to get Git access to projects outside the group and its ancestors.

Non-goals

This document focuses on providing core functionality for supporting authentication via SSH certificates. The potential improvements are described in Follow Ups.

Proposal

MVC

A group admin generates an SSH key pair to be used as a Certified Authority file (ssh-keygen -f CA):

- The private key is used to issue user certificates
- The public key is added to a group in order to grant access to the group via the user certificates

User certificate

A group admin issues user certificates using CA private key and specifies either a GitLab username or a user's primary email as the key identity:

shell

```
ssh-keygen -s CA -I user@example.com -V +1d user-key.pub
```

As a result, a user certificate of the following structure is generated:

shell

```
ssh-keygen -L -f sshhosted25519key-cert.pub
```

sshhosted25519key-cert.pub:

Type: ssh-ed25519-cert-v01@openssh.com user certificate

Public key: ED25519-CERT SHA256:dRVV49XJHt85X1seqr9xXyxyuuGTbtFV6LbwlrX6BIQ

Signing CA: RSA SHA256:UAcgUeGoXrs8WOT/N+bmqY2vB9145Mc5NaN1Y977NCI (using rsa-sha2-512)

Key ID: "user@example.com"

Serial: 1

Valid: from 2023-07-31T18:20:00 to 2023-08-01T18:21:34

Principals: (none)

Critical Options: (none)

Extensions:

permit-X11-forwarding

permit-agent-forwarding

permit-port-forwarding

permit-pty

permit-user-rc

- Type is the type of the user certificate. Only user certificates are accepted by GitLab Shell, all other types are rejected.

- Public Key is the public key of the user.

- Signing CA is the public key of CA. Its fingerprint is used to find a group associated with the user certificate.

- Key ID is a user's username or primary email. It is used to associate a GitLab user with the user certificate.

- Serial is a serial number of the user certificate. It can be used to distinguish between different certificates created by the same CA.

- Valid indicates the period of validity. This value is validated by GitLab Shell: expired and

not yet valid user certificates are rejected.

- Principals, Critical Options and Extensions are used to embed additional information into the user certificate. This fields can be potentially used in future iterations to apply additional restrictions on a user certificated.

Application behavior

GitLab Shell is the project responsible for handling commands sent to GitLab instance via SSH. When a user tries to establish an SSH connection and authenticate via a public key, GitLab Shell sends an internal API request to /authorizedkeys endpoint to detect whether the key is associated with a GitLab user. If a certificate is used for authentication, GitLab Shell can recognize it and perform a request to /authorizedcerts instead.

1. A group admin adds CA.pub file to a group.
1. A user tries to authenticate using a certificate signed by the CA.
1. GitLab Shell sends to /authorizedcerts the fingerprint of the CA and the user identity (either GitLab username or the primary email).
1. GitLab Rails finds a group that has a CA.pub with the fingerprint added and the user. The CA.pub is unique for an instance by fingerprint which defines the one-to-one relationship between CA and a group.
1. GitLab Shell remembers the namespace full path for the established connection.
1. GitLab Shell sends a request to /allowed endpoint every time a check whether a user has access to a particular project is needed. The namespace full path is passed to /allowed endpoint.
1. GitLab Rails checks whether the namespace matches the project namespace or one of its ancestors in order to determine whether a user has access to this project via the certificate.
1. If all the checks above are successful, the user gets access to the project.

mermaid

sequenceDiagram

```
User->>+GitLab Shell: Auth using SSH Certificate
GitLab Shell->>+GitLab Rails: /authorizedcerts?key=fingerprint-of-signing-
key&useridentity=username-or-primary-email
GitLab Rails-->>-GitLab Shell: responds with the namespace that configures the CA and the
username of the user
GitLab Shell-->>User: Authenticated successfully
User->>+GitLab Shell: Git command to a specific project
GitLab Shell->>+GitLab Rails: /allowed [namespace=namespace]
GitLab Rails-->>-GitLab Shell: responds that the user is allowed to access a project in
this namespace
GitLab Shell-->>User: success
```

Examples

1. Access a project outside the group that configures CA.pub.

Given there is the following hierarchy of groups

plaintext

a/b/c/d/e/f
|
./g/h/i

- A group admin adds CA.pub to d and a user is authenticated using a certificate signed by the CA.
- When a user clones a/b/c/d/e/f/project, we send a/b/c/d/e/project project full path and a/b/c/d namespace full path: the user is allowed to clone the project because d is an ancestor of the project's namespace.
- When a user clones a/b/c/g/h/i/project, the user is not allowed to clone the project because d is not in a list of its ancestors.

1. A group that configures CA.pub is transferred to a different namespace.

The existing certificates are still valid because the namespace full path is stored per connection. When a user reconnects, another request to /authorizedcerts is sent and the new full path of the namespace is returned.

Open questions

Multiple SSH certificates to different projects

A user may have different SSH certificates to access different projects. When the user establishes an SSH connection, the SSH client will iterate over a number of potential options in order to find the one that authenticates successfully. With the current architecture, the first certificate that provides access to a namespace is accepted, even if the user is meant to get access to a different project.

For example:

1. A user has valid certificates for a and b groups.
1. The user is successfully authenticated using a.
1. The user tries to clone b/project and fails.

A workaround for this scenario is to configure Git to use a particular certificate during an SSH connection. Add the following to your .gitconfig file:

```
plaintext
[core]
  sshCommand = ssh git@gitlab.com -i cert.pub
```

A single certificate cannot be revoked

Revocation of a single user certificate is out of the scope of this MVC. It is possible to implement this feature but the feasibility should be discussed.

Supporting it will complicate the implementation and UI/UX. However, the risk of a compromised certificate can be significantly reduced by the following actions:

- Expiration date of a user certificate. It should be highly recommended in the docs.
- Rotation of the CA which revokes all the current user certificates.
- Implementing source-address feature that allows restricting IP address(es) from which the user certificate can be used.

A certificate can be used across multiple GitLab instances

The information about GitLab instance is not embedded into a user certificate. It means that it can be used across multiple GitLab instances as long as the KeyId value is recognized in those instances.

Potential solution:

- An optional field that restricts usage of a user certificate to a particular instance can be implemented using Extensions in a follow-up. Specifying extension:login@gitlab.com=username is a more secure and flexible option, but we can support both.

CA cannot be reused by multiple groups

The CA.pub fingerprint must be unique and cannot be reused by multiple groups. The one-to-one relationship is chosen by design to be able to find a single group that a user has access to.

Another option is to embed the namespace into the user certificate using Extensions or Critical Options.

Pros:

- CA can be reused by multiple groups.
- A user certificate tells which group to get access to rather than asks which group it can get access to.

Cons:

- Requires a custom requirement to the user certificate format.
- If some other group adds CA.pub, a user may unintentionally get access to that group.

Potential solution:

- An optional field that restricts usage of a user certificate to a particular group can be implemented using Extensions or Critical Options in a follow-up. CA still cannot be reused but a user certificate cannot be used for some other group.

Iteration plan

Component	Milestone	Group	Changes
-----	-----	-----	-----
GitLab Shell	16.3	Source Code	Implement

authentication using SSH certificates in GitLab Shell |
| GitLab Rails | 16.4 | Source Code | Implement the
internal GitLab Rails API endpoint authorizedcerts to find a group that configures the CA.pub |
| GitLab Rails | 16.4 | Source Code | Implement a GitLab
Rails API endpoint for groups to add/remove a CA.pub |
| GitLab Rails | Next 2-3 milestones | Authentication and Authorization | Implement Group
Settings UX to add/remove CA.pub |
| GitLab Rails | Next 2-3 milestones | Authentication and Authorization | Implement an option
to enforce using SSH certificates only for authentication and forbid personal SSH keys and
access tokens |

Follow-ups

The functionality that is related to the topic but out of the scope of this blueprint:

- Enforce using SSH certificates only for authentication by disabling Git over HTTPS for an instance or a Group: a must-have functionality.
- Enforce using Group-level SSH keys only by forbidding the use of personal SSH keys for an instance or a Group: a must-have functionality.
- Specifying source-address Critical Option that restricts usage of a user certificate to a set of IP addresses: a nice-to-have functionality.
- Specifying login@hostname=username Extensions that restrict usage of a user certificate to a set of instances: a nice-to-have functionality.
- Signing commits using SSH certificates: a nice-to-have functionality.
- Revoke a single user certificate. It requires complex UI/UX while the risk can be significantly mitigated by using other features.

SECTION: engineering/architecture/design-documents/tailwindcss/_index.md

{{< engineering/design-document-header >}}

Summary

Styling elements in GitLab primarily relies on CSS utility classes. Those are classes that generally define a single CSS property and that can be applied additively to change an element's look.

We have developed our own tooling in the GitLab UI project to generate the utils we need, but our approach has demonstrated a number of flaws that can be circumvented by delegating that task to the Tailwind CSS framework.

This initiative requires that we deprecate existing utilities so that Tailwind CSS can replace them.

Motivation

In June 2019, we have consolidated our usage of CSS utility classes through RFC4 which introduced the concept of silent classes, where utilities would be generated from a collection

of manually defined SCSS mixins.

This has served us well, but came with some caveats:

- Increased development overhead: whenever a new utility is needed, it has to be manually added to the GitLab UI project. One then needs to wait on a new version of @gitlab/ui to be released and installed in the consumer project.
- Inconsistencies: Without any tooling in place to check how utilities are named, we have seen many inconsistencies make their way in the library, making it quite unpredictable. The most striking example of this was the introduction of desktop-first utilities among a majority of mobile-first utils, without any way of distinguishing the former from the latter, other than looking at the source.
- Disconnection between the utilities library and its consumers: When a utility is added to the library, it is made available to any project that uses @gitlab/ui. As a result, some utils are included in projects that don't need them. Conversely, if all consumers stop using a given util, it could potentially be removed to decrease the CSS bundle size, but we have no visibility over this.
- Limited autocompletion: Although it's possible to configure autocomplete for the existing library, it is restricted to the utilities bundle. In contrast, Tailwind CSS autocomplete aligns with an on-demand approach, ensuring that all utilities are readily available. Additionally, IDE extensions can enhance understanding by revealing the values applied by a specific utility.

As part of this architectural change, we are alleviating these issues by dropping our custom built solution for generating CSS utils, and delegating this task to Tailwind CSS.

It is worth noting that this was previously debated in RFC107.

The RFC was well received. The few concerns that were raised were about the CSS utility approach as

a whole, not the way we implemented it. This initiative's purpose is not to question our reliance

on utility classes, but to consolidate its implementation to improve engineers' efficiency when working with CSS utils.

Why Tailwind CSS?

Here are a few reasons that led us to choosing Tailwind CSS over similar tools:

- It is a long-standing project that has been battle-tested in many production apps and has a healthy community around it.
- Tailwind CSS is well maintained and keeps evolving without getting bloated.
- It integrates well in all of our tech stacks
 - Ruby on Rails projects can leverage the tailwindcss-rails Gem.

- Nuxt apps can set up the tailwindcss module.
- More generic frontend stacks can use the tailwindcss Node module.

Goals

This blueprint's goal is to improve the developer experience (DX) when working with CSS utility classes.

As a result of this initiative, frontend engineers' efficiency should be increased thanks to a much lower development overhead.

Non-Goals

As stated in the motivations above, this focuses on improving an existing architectural decision, not on replacing it with a new design. So this therefore:

- Is not aimed at revisiting the way we write CSS or how we apply styles within our projects.
- Does not focus on user-facing improvements. This change is mostly a developer experience enhancement.

The resulting increase in efficiency could certainly indirectly improve user experience, but that is not our primary intent.

Proposal

We will be setting up Tailwind CSS in GitLab UI and GitLab. The intent is to have the main Tailwind CSS configuration in GitLab UI. This step is where we'll be maintaining the Pajamas-compliant configuration properties (color, spacing scale, etc.). The Tailwind CSS setup in GitLab will inherit from GitLab UI's setup. The subtlety here is that, in GitLab, we will be scanning both the GitLab codebase and the @gitlab/ui Node module. This will ensure that GitLab UI does not need to expose any CSS utilities anymore, but the ones it relies on are still generated in GitLab. A similar setup will need to be introduced in other projects that use CSS utilities and need to upgrade to the Tailwind CSS-based version.

Pros

- We are removing the cumbersome workflow for adding new utilities. One should be able to use any utility right away without contributing to another project and waiting through the release cycle.
- We are introducing a predictable library, where the naming is decided upon in the overarching Tailwind CSS project. As engineers know, naming things is difficult, and it's best that we defer this to a well-established project.

- Engineers should be able to refer to Tailwind CSS documentation to know what utils are available and how to use them. No need to read through GitLab UI's source code anymore.
- Because Tailwind CSS generates the required utils by scanning the consumer's codebase, we'll be sure to only generate the utilities we actually need, keeping CSS bundle sizes under control. This must be taken with a grain of salt though: Tailwind CSS is extremely flexible and makes it possible to generate all sorts of utils, sometimes with developer-defined values, which could result in large utils bundles depending on how we'll adopt Tailwind CSS' features.
- We'll benefit from a robust IDE integration providing auto-completion and previews for the utils we support.

Cons

- More setup: each project that requires CSS utils would need to have Tailwind CSS set up, which might be more or less tedious depending on the environment.
- One more dev dependency in each project.
- Inability to use string interpolation to build class names dynamically (Tailwind CSS needs to see the full names to generate the required classes).
- We'll need a migration: we'll need to ensure usages of the existing CSS utilities library don't break, which implies a deprecation/migration process.

Design and implementation details

In order to prevent breakages, we are taking an iterative approach to moving away from the current library. The proposed path here is purposefully rough around the edges. We acknowledge that it's not a one size fits all solution and that we might need to adjust to some cases along the way.

Here's the basic process:

1. The base Tailwind config is defined in `@gitlab/ui` and exported as `tailwind.defaults.js`. This config defines breakpoints, colors, spacing, font size, and other configuration that should be consistent across all projects.
1. `gitlab-org/gitlab` has a `tailwind.config.js` file that uses `@gitlab/ui/tailwind.defaults.js` as a preset so all configuration is inherited. The content property is set to scan Vue, JS, HAML, and Ruby files in `gitlab-org/gitlab` and `@gitlab/ui`.
1. `scripts/frontend/tailwindalltheway.mjs` compares Tailwind utilities to `@gitlab/ui` utilities. The utilities that are not a perfect match are added to `config/helpers/tailwind/cssinjs.js` in `gitlab-org/gitlab`. The utilities that are a perfect match are automatically generated by Tailwind.
1. `config/helpers/tailwind/cssinjs.js` is imported in `tailwind.config.js` and uses the `addUtilities` function to register these utilities.
1. Legacy utility usages in `gitlab-org/gitlab` are iteratively migrated to their Tailwind equivalents. This causes `config/helpers/tailwind/cssinjs.js` to shrink.

1. Legacy utility usages in @gitlab/ui are migrated to their Tailwind equivalents. When no legacy utility usages remain in @gitlab/ui, config/helpers/tailwind/cssinjs.js is empty and can be removed along with the supporting tooling.
1. All SCSS utility mixins are removed from @gitlab/ui and a major version is released.

SECTION: engineering/architecture/design-documents/track_vulns_across_branches/_index.md

{{< engineering/design-document-header >}}

Summary

Perhaps one of the most in demand Sec features for GitLab currently is the ability to track vulnerabilities across multiple branches. While the current implementation of vulnerability management features in GitLab offers a lot of power, the implementation is very inflexible. As a result, workflows which may involve running different versions of an application, be that old releases or modified versions are unable to track vulnerabilities without forking the code to separate projects entirely.

Unfortunately, GitLab's current code is not built to facilitate t